

Portfolio Optimization: Markowitz Mean-Variance Optimization

Presented by: Anjali Tomerlin

October 3, 2026

Key terms

- **Asset:** Investment resource that can be bought and sold
- **Short Selling / Shorting:** Selling an asset we do not currently own
- **Portfolio:** The collection of assets we own

Brief Interlude on Shorting

- Say we go to a company that has N of Stock A. Say the brokerage sells $k, k \leq N$, of that stock from the company on our behalf, then our account will have $-k$ of Stock A, assuming we start with 0 of Stock A (a.k.a. we are **SHORT** by k of Stock A).
 - Notice that we are counting debt in terms of Stock A, not dollars
- You will now immediately sell the k of Stock A that you borrowed. Let's say from this sale you made x_1 dollars.
- Eventually, you will have to buy back that k of Stock A that the company lent to you to set your account back to 0 of Stock A. Let's say we buy back the stock for the current market value of x_0 dollars.
- If $x_0 < x_1$ then we have made a profit!

Putting it in Mathematical Terms

Suppose we buy asset for x_0 dollars and sell it for x_1 dollars, we define the following:

(Gross) Return on Asset

$$R = \frac{x_1}{x_0}$$

Rate of Return on Asset

$$r = \frac{x_1 - x_0}{x_0} = R - 1$$

Building Our Portfolio

Now let's say we have a portfolio consisting of n assets and an initial budget of b dollars that we wish to allocate to these assets. We also assign each asset a weight of w_i (where a negative weight means we are shorting that asset in our portfolio).

- Amount of budget we assign to asset i is $b_i = bw_i$
- To preserve our budget constraint, we require that $\sum_{i=1}^n w_i = 1$, so $\sum_{i=1}^n b_i = b$
- If we short some stocks, we allow ourselves more funds to allocate to other assets
- For the sake of simplicity, this model assumes unrestricted short selling and no transaction costs, borrowing costs, etc.

Returns On Our Portfolio

Let $x_{1,i}$ be the amount we make from selling asset i and R_i be the return on investment i , then $x_{1,i} = R_i b_i = R_i b w_i$. Summing these for each asset, then dividing by the total budget b gives:

Total Return on Portfolio

$$R = \sum_{i=1}^n R_i w_i$$

We also know that the rate of return on asset i is $r_i = R_i - 1$. This gives us:

Rate of Return on Portfolio

$$r = R - 1 = \sum_{i=1}^n R_i w_i - \sum_{i=1}^n w_i = \sum_{i=1}^n w_i r_i$$

Defining the Problem

If we consider the rates of returns on each asset r_i as random variables. We can then choose the weighting factors $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ such that the following goals are accomplished simultaneously:

- The portfolio achieves a desired baseline expected rate of return
- The portfolio has minimum volatility (i.e. the standard deviation, or equivalently the variance, of the rate of return on the portfolio is at a minimum)

These are certainly not the only goals possible. Sometimes we want maximum rate of return on our portfolio while constraining the volatility (we will get to that!). Can you think of other possible goals, constraints, or additions to the model?

Putting the Problem in Math Terms

We define the following relevant variables:

- The random vector of rates of returns $\mathbf{z} = [r_1, r_2, \dots, r_n]^T$
- The expected rates of return for a given asset $\mu_i = E(r_i)$
- The vector of expected rates of returns $\mathbf{m} = [\mu_1, \mu_2, \dots, \mu_n]^T$
- The covariance matrix $\mathbf{S} = \text{Cov}(\mathbf{z})$, where $S_{i,j} = \text{Cov}(r_i, r_j)$. Note that diagonal entries $S_{i,i} = \text{Var}(r_i)$ and $\mathbf{S}$ is symmetric since $\text{Cov}(r_i, r_j) = \text{Cov}(r_j, r_i)$

Putting the Problem in Math Terms Continued

Since r_i are random variables, $r = \sum_{i=1}^n w_i r_i = \mathbf{w}^T \mathbf{z}$ is a random variable as well. The following can be observed:

Mean of r

$$\mathbf{m}^T \mathbf{w} = \sum_{i=1}^n w_i \mu_i$$

Variance of r

$$\mathbf{w}^T \mathbf{S} \mathbf{w}$$

Explaining the Variance of r

The mean of r is pretty clear to see (use the linearity of expectation), but the variance $\mathbf{w}^T \mathbf{S} \mathbf{w}$ can be a bit more difficult to intuit. Let's work through it:

$$\begin{aligned}\mathbf{S} \mathbf{w} &= \left[\sum_{i=1}^n w_i \text{Cov}(r_1, r_i), \sum_{i=1}^n w_i \text{Cov}(r_2, r_i), \dots, \sum_{i=1}^n w_i \text{Cov}(r_n, r_i) \right]^T \\ \mathbf{w}^T \mathbf{S} \mathbf{w} &= w_1 \sum_{i=1}^n w_i \text{Cov}(r_1, r_i) + w_2 \sum_{i=1}^n w_i \text{Cov}(r_2, r_i) + \dots + w_n \sum_{i=1}^n w_i \text{Cov}(r_n, r_i) \\ &= \sum_{j=1}^n \sum_{i=1}^n w_j w_i \text{Cov}(r_j, r_i) = \text{Var} \left(\sum_{i=1}^n w_i r_i \right) = \text{Var}(r)\end{aligned}$$

Our Mathematical Problem: Minimizing Risk

Remembering back to our 2 goals (achieving a baseline expected rate of return μ_b and minimizing the variance of r), we get the following optimization problem:

$$\begin{aligned} &\text{minimize } \frac{1}{2} \mathbf{w}^T \mathbf{S} \mathbf{w} \\ &\text{subject to the constraints } \mathbf{m}^T \mathbf{w} \geq \mu_b \text{ and } \mathbf{1}^T \mathbf{w} = 1 \end{aligned}$$

- Note that $\mathbf{1} = [1, 1, \dots, 1]^T$ (the vector of all ones), so the last constraint is just requiring that our weights add to 1
- Recall, we are requiring our weights add to one to preserve our budget constraint
- Why the $1/2$? Quite simply, it makes the math nice. Taking constant multiples like this still preserves the location of all critical points and extrema.

If you've taken Calculus 3, this should invoke some memories of **Lagrange Multipliers!**

Is This The Only Problem Out There?

No, absolutely not! There are **so many** possible optimization problems you can construct from the variables we just defined. Let's consider another possible goal with the following stipulations:

- The portfolio maximizes expected rate of return
- The portfolio does not exceed a certain volatility threshold (i.e. the variance of the rate of return on the portfolio is bounded)

A Second Problem: Maximizing Reward!

Let's put these 2 goals (maximizing expected return and restraining the variance r to be less than $2\sigma_{max}^2$) into mathematical terms:

$$\begin{aligned} &\text{maximize } \mathbf{m}^T \mathbf{w} \\ &\text{subject to the constraints } \frac{1}{2} \mathbf{w}^T \mathbf{S} \mathbf{w} \leq \sigma_{max}^2 \text{ and } \mathbf{1}^T \mathbf{w} = 1 \end{aligned}$$

- We are simply swapping which aspect of our portfolio we prioritize (risk, measured by the variance of r , or return, measured by the expected value of r)
- Just to reiterate, the $1/2$ factor on the variance part is just a convention in quadratic programming, one that you will see when using industry-standard packages like `cplex` (which we will get into soon)!

How Can We Solve These Optimization Problems?

Although Lagrange Multipliers (the typical definition you learn in Calculus 3, that is) are quite useful, they are only applicable when we have only equality constraints. Here, we have inequality constraints as well.

The specific kind of problem we are dealing with here is known as **convex optimization problems**. We can solve these sorts of problems using packages in Python such as `cvxpy`.

- The mathematical reason why these problems are considered convex comes down to the fact that covariance matrices are always positive semi-definite.
- Before we can go ahead to solve these kinds of optimizations problems using code, let's first understand where our variables come from, starting with our covariance matrix.

How Do We Find Our Covariance Matrix?

Short answer: data! Suppose we have the following $s \times n$ historical data matrix on rates of returns for our n assets of interest:

$$\mathbf{X} = \begin{array}{ccccc} & & \text{Asset 1} & \text{Asset 2} & \cdots & \text{Asset } n \\ \text{Time 1} & & r_{1,1} & r_{1,2} & \cdots & r_{1,n} \\ \text{Time 2} & & r_{2,1} & r_{2,2} & \cdots & r_{2,n} \\ & \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{Time } s & & r_{s,1} & r_{s,2} & \cdots & r_{s,n} \end{array}$$

Each row of this matrix represents a point in time where the rates of return for each of our n assets of interest were sampled / observed. Here, we have a sample size of s .

How Do We Find Our Covariance Matrix?

Using this data, we can calculate an estimate for our true covariance matrix:

$$\hat{\mathbf{S}} = \frac{1}{s-1} \mathbf{X}_c^T \mathbf{X}_c$$

Where $\mathbf{X}_c$ is the mean-centered data matrix $\mathbf{X}$:

$$\mathbf{X}_c = \begin{array}{cc} & \begin{array}{cccc} \text{Asset 1} & \text{Asset 2} & \cdots & \text{Asset } n \end{array} \\ \begin{array}{c} \text{Time 1} \\ \text{Time 2} \\ \vdots \\ \text{Time } s \end{array} & \begin{bmatrix} r_{1,1} - \bar{r}_1 & r_{1,2} - \bar{r}_2 & \cdots & r_{1,n} - \bar{r}_n \\ r_{2,1} - \bar{r}_1 & r_{2,2} - \bar{r}_2 & \cdots & r_{2,n} - \bar{r}_n \\ \vdots & \vdots & \ddots & \vdots \\ r_{s,1} - \bar{r}_1 & r_{s,2} - \bar{r}_2 & \cdots & r_{s,n} - \bar{r}_n \end{bmatrix} \end{array}$$

Where $\bar{r}_i$, $1 \leq i \leq n$, is the mean of the i th column, calculated as $\bar{r}_i = \frac{1}{s} \sum_{j=1}^s r_{j,i}$

Math Stuff: Positive Definiteness in Context

S is NOT positive definite

- There are many reasons for a covariance matrix to not be PD, but a quite common one is that our covariance matrix was calculated from **too small of a sample**.
- A sample covariance matrix $\hat{\mathbf{S}}$ of size $n \times n$ calculated from a data matrix $\mathbf{X}$ of size $s \times n$ has maximum rank $\min(s - 1, n)$.
- Thus, $s \geq n + 1$ is necessary for $\hat{\mathbf{S}}$ to be full rank, but it is not sufficient.

S is positive definite

- This requires that no two of our assets are perfectly correlated.
- This requires that none of our assets have zero variance (no risk-free assets).
- This requires that the sample size of the data used to calculate our covariance matrix estimate $\hat{\mathbf{S}}$ was sufficiently large ($s \geq n + 1$, where n is the number of assets).
- Among other conditions, this is **NOT** an exhaustive list.

Why Does All This "Definite" Stuff Even Matter?

In short, our covariance matrix being positive definite guarantees a **unique** solution to our optimization problem.

If our covariance matrix (for whatever reason, whether that be too small of a sample or zero variance assets) is not positive definite then uniqueness is not guaranteed and we are able to get an **infinite** number of valid solutions to our optimization problem!

- This is a problem because we start to ask ourselves "which weights should I go with then?" The math behind that question is complicated and beyond the scope of this workshop (left as an exercise to the reader, you could say)!
- Not to worry though, most convex optimization solvers (like `cvxpy` in Python) can handle this problem using some clever and advanced techniques!

One Last Thing: Estimating Our Expected ROR Vector

This one is easy, we will estimate $\mathbf{m}$ as the column means of our data that we defined before:

$$\hat{\mathbf{m}} = \begin{bmatrix} \bar{r}_1 \\ \bar{r}_2 \\ \vdots \\ \bar{r}_n \end{bmatrix}$$

Where $\bar{r}_i, 1 \leq i \leq n$, is the mean of the i th column of our matrix data. Note that this is a simple but naive estimator for $\mathbf{m}$. In practice, expected returns are difficult to estimate and a major source of error in MVO.

Putting It Into Code

Now it's time to implement our optimization problems using the `cvxpy` package in Python! I encourage you all to read the documentation of `cvxpy` to understand how exactly it goes about solving these problems, especially if you are more mathematically inclined.

Steps for implementation:

1. Get / import our historical rate of return data.
2. Use our data to calculate an estimated covariance matrix $\hat{\mathbf{S}}$.
3. Calculate our vector of estimated expected rates of returns $\hat{\mathbf{m}}$.
4. Plug our estimates $\hat{\mathbf{S}}$ and $\hat{\mathbf{m}}$ into the optimization problem we defined, and solve for our optimal set of weights $\mathbf{w}^*$ using `cvxpy`'s powerful solver.

Coding Markowitz MVO from Scratch in Colab



[https://colab.research.google.com/drive/
15XFC1p3RC5h79npHXsT3HBG6bxiLl7gd?usp=sharing](https://colab.research.google.com/drive/15XFC1p3RC5h79npHXsT3HBG6bxiLl7gd?usp=sharing)

<https://bit.ly/4AFP54b>

GitHub With All Workshop Materials



<https://github.com/dsiufl/WorkshopArchive>

Thank you for participating!



References

Burke, James V. Markowitz Mean-Variance Portfolio Theory. University of Washington.
<https://sites.math.washington.edu/~burke/crs/408/fin-proj/mark1.pdf>.